Comparative Evaluation of the Red Deer Algorithm for Load Balancing in Cloud Computing

Krish Sharma[1], Tanmay Tushar[1], Nripendranarayan Das[1*], Ashish Jain[2*]

[1]Department of Information Technology, Manipal University Jaipur, Jaipur, 303007, Rajasthan, India.
[2]Department Name, College Name, Jaipur, Rajasthan, India.

*Corresponding author(s). E-mail(s): nripendranarayan.das@jaipur.manipal.edu; ashishjn.research@gmail.com;

Abstract

This paper investigates the critical challenge of load balancing in cloud computing, a key factor influencing the performance, reliability, and efficiency of cloud based services. With the rise of cloud computing technologies, the need for effective workload distribution across heterogeneous and geographically distributed resources has become increasingly complex. Load balancing is categorized as an NP-hard problem due to its vast solution space, necessitating the use of metaheuristic algorithms for efficient problem solving. In this study, we propose a Red Deer Algorithm (RDA)-based load balancing approach for cloud task scheduling in heterogeneous cloud environments using a multi-objective QoS-aware fitness function. Additionally, a comparative performance analysis is conducted with multiple metaheuristic scheduling algorithms including GA, PSO, GWO, and WOA using metrics such as makespan, response time, and resource utilization. Simulation results using CloudSimPlus demonstrate that the RDA based method significantly reduces costs and response time. The proposed approach is further evaluated using convergence analysis, parameter sensitivity analysis, and statistical validation through confidence intervals, t-tests, and Wilcoxon signed-rank tests. The findings demonstrate that the proposed RDA-based model achieves competitive scheduling performance while improving QoS and resource utilization across different cloud scheduling scenarios.

Keywords: Metaheuristic Algorithms, Red Deer Algorithm, Cloud Computing, Grey Wolf Optimization, Genetic Algorithm, Particle Swarm Optimization, Whale Optimization Algorithm, Load Balancer, Virtual Machines

1 Introduction

Cloud computing is a model for delivering resources such as servers, storage systems, databases, networking facilities, software applications, and analytics tools over the Internet, commonly referred to as "the cloud" [1]. The adoption of cloud computing enables users to access applications from anywhere in the world. This approach differs significantly from traditional hosting models due to two key entities: the Cloud Service Provider (CSP) and the Cloud Service User (CSU) [2]. In most cases, the use of cloud services involves associated costs.

Cloud computing services typically follow a "pay-as-you-go" and "on-demand" model, allowing customers to avoid long-term contractual commitments. The self-service capability enables users to provision resources automatically without requiring manual intervention, sales processes, or complex service setup procedures. This automation simplifies service acquisition and enhances usability while contributing to competitive pricing [3].

Due to the dynamic and distributed nature of cloud computing environments, where resources are provisioned and consumed over the Internet as needed, effective resource management is essential. As the number of applications and users that rely on cloud services continues to grow, it becomes increasingly important to ensure that workloads are efficiently distributed across available computing nodes. This distribution helps maintain system responsiveness, reliability, and overall performance. Load balancing plays a critical role in achieving this objective [4].

Load balancing refers to the efficient and equitable distribution of tasks across computing resources, such as virtual machines (VMs), to maintain optimal resource utilization and meet system performance requirements (e.g. processor load, primary and secondary memory usage, latency and network load) [5]. When certain VMs experience higher workloads than others, overall system efficiency decreases. Load balancing addresses this issue by allowing intelligent distribution of tasks among available computing resources [6].

The primary objective of load balancing is to maintain acceptable levels of resource utilization measured in terms of CPU usage, memory consumption, response time, and network traffic while satisfying user requirements and maximizing resource efficiency [7]. An effective load balancing mechanism considers both the current state of resource availability and the specific requirements of the incoming tasks to make informed allocation decisions. This approach helps reduce traffic bottlenecks, improve system performance, increase throughput, and ensure optimal utilization of cloud infrastructure. In this work, the proposed scheduling framework is implemented and evaluated using CloudSim Plus, which provides improved scalability, modularity, and simulation support compared to earlier CloudSim versions.

Therefore, the choice of load balancing strategy is of critical importance [8]. Traditional deterministic or rule-based algorithms often struggle to handle the scale, complexity, and dynamic nature of cloud environments. In contrast, metaheuristic algorithms have emerged as promising solutions for load balancing problems, as they can efficiently explore large solution spaces and identify near optimal solutions within reasonable computational time [9].

A metaheuristic solution can be defined as an interactive set of processes that govern the exploration and exploitation of the search landscape [10, 11]. Metaheuristic methods are one of the strategies that can be applied to resolve performance issues, for example, task scheduling [12, 13]. Metaheuristic algorithms are high-level problem-independent methods that guide lower level heuristics to finding optimal or near-optimal solutions to complicated optimization problems. These algorithms, e.g., Genetic Algorithms (GA), Particle Swarm Optimization (PSO) [14], and Ant Colony Optimization (ACO) [15], are inspired by natural phenomena like evolutionary processes, collective behavior, or reproduction strategies of certain animal species [10, 16]. Even though they do not guarantee the achievement of a global optimum, such approaches are well adapted to providing acceptable solutions for NP-hard problems, especially if exhaustive search is computationally infeasible. Their flexibility, scalability, and adaptability make them particularly well adapted to dynamic environments like cloud computing, where conditions change constantly and optimum task-resource assignment needs constant re-tuning [17].

Although several metaheuristic algorithms have been applied to cloud scheduling problems, many existing approaches focus on a limited set of QoS objectives or assume simplified cloud environments with homogeneous resources. In addition, several studies lack detailed scalability analysis, robustness evaluation, and statistical validation of results. These limitations motivate the need for a scheduling approach capable of handling heterogeneous cloud infrastructures while maintaining balanced optimization of multiple QoS parameters.

Currently, the selection of the most suitable cloud services is optimized using QoS criteria, which helps address challenges related to connectivity and the definition of QoS parameters [19]. QoS remains one of the most critical aspects in addressing challenges within cloud computing. In the present study, an RDA-based load balancing approach is proposed for cloud task scheduling using a multi-objective QoS-aware fitness function that considers makespan, response time, resource utilization, execution cost, and load balancing during task allocation. RDA draws inspiration from the natural mating behavior of Red Deer [18]. The algorithm begins by initializing a population of Red Deer, which are then classified into male and female individuals. All male Red Deer update their positions and then mate. After all mating operations, a new generation is formed by selecting offspring based on fitness, and the process iterates until a predefined number of generations is reached [18]. This biologically inspired structure allows RDA to effectively balance exploration and exploitation, making it highly suitable for solving complex optimization problems such as load balancing in cloud computing, where dynamic resource allocation and efficiency are essential [20]. The proposed scheduling framework further considers heterogeneous virtual machine configurations with varying processing capacities, availability levels, and execution costs in order to better represent realistic cloud computing environments.

The main contributions of this work are summarized as follows:

- An RDA-based cloud task scheduling approach is proposed for heterogeneous cloud environments.

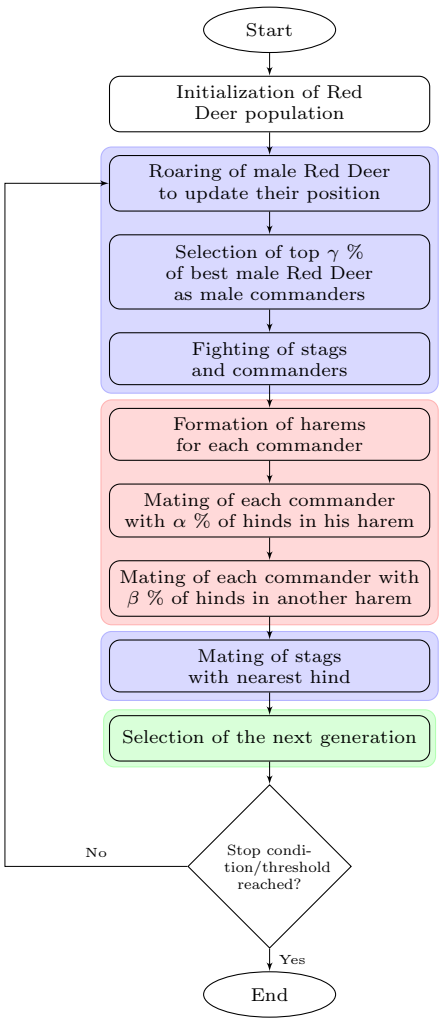


Fig. 1: Flow Chart for Workflow of RDA [18]

- A multi-objective QoS-aware fitness function is designed to optimize makespan, response time, execution cost, resource utilization, and load balancing simultaneously.
- The proposed model is evaluated under heterogeneous VM configurations and varying workload scales using CloudSim Plus.
- Extensive experimental analysis including convergence analysis, parameter sensitivity analysis, scalability analysis, robustness evaluation, and statistical validation is performed.
- The proposed approach is benchmarked against multiple metaheuristic scheduling algorithms including GA, PSO, GWO, and WOA.

The remainder of this paper is organized as follows: Section 2 discusses related work in the domain. Section 3 presents the proposed model. The development of the RDA in accordance with the proposed model is detailed in Section 4. Section 5 outlines the experimental setup, while Section 6 presents the results and analysis. Finally, Section 7 concludes the paper with future research directions.

2 Related Work

Goyal and Singh [21] proposed the Ant-based Load Balancing Algorithm (ALBA) for dynamic load balancing in grid computing environments. Unlike traditional ACO, in which pheromone trails are deposited on paths between nodes, ALBA associates pheromone directly with resources to indicate their current load levels. When a task completes successfully on a resource, its pheromone value is increased to signal light load, whereas task failure triggers a pheromone decrease to signal heavy load. Task assignment is then guided by transition probabilities computed from these pheromone values, directing incoming tasks toward the least loaded resources. The algorithm was evaluated using GridSim across two scenarios: varying the number of tasks from 25 to 100 with a fixed resource pool of 10, and varying the number of resources from 10 to 30 with a fixed task count of 75. Results demonstrated that ALBA achieved significantly more uniform resource utilization compared to the baseline without load balancing (WALBA), with a standard deviation of load distribution not exceeding 0.77, compared to a range of 2.35 to 3.18 for WALBA. However, several critical limitations substantially restrict the practical applicability of this work. First, and most fundamentally, ALBA is designed and evaluated exclusively for grid computing, not cloud computing. The architectural assumptions of the paper, including geographically distributed independent resources and static resource pools, differ significantly from the VM-based, virtualized, and elastically scalable nature of cloud environments, making direct applicability to cloud load balancing questionable. Second, the comparative evaluation is conducted solely against WALBA, a trivial baseline that performs no load balancing whatsoever. The absence of comparison against other established metaheuristic approaches such as PSO, GA, or other ACO variants makes it impossible to objectively position ALBA's performance within the broader literature. Third, the evaluation is limited to a maximum of 100 tasks and 30 resources, which represents a very small scale relative to real-world grid or cloud deployments, raising serious concerns about scalability. Fourth, the study does not evaluate critical QoS metrics such as makespan, task response time, energy consumption, or fault tolerance, reporting only resource utilization and standard deviation of load distribution. This narrow evaluation scope makes it difficult to assess the algorithm's overall effectiveness as a scheduling solution.

Kruekaew and Kimpan [22] proposed the Heuristic Task Scheduling with Artificial Bee Colony (HABC) algorithm, which combines swarm intelligence from the ABC algorithm with heuristic scheduling strategies, namely First Come First Serve (FCFS), Smallest Job First (SJF), and Largest Job First (LJF), to optimize VM scheduling and load distribution in cloud computing environments. The proposed method was evaluated against ACO, PSO, and IPSO using the CloudSim simulation toolkit across

twelve datasets spanning four distribution types: random, normal, right-skewed, and left-skewed. Experimental results demonstrated that HABC with the LJF heuristic consistently achieved the lowest minimum, maximum, and average makespan values, as well as the lowest degree of load imbalance, across both homogeneous and heterogeneous environments. Notably, the standard deviation of VM processing times under HABC was substantially lower than that of ACO, PSO, and IPSO, indicating a more balanced workload distribution. However, several critical limitations warrant attention. First, the evaluation was conducted exclusively in a simulated environment using CloudSim, and the authors themselves acknowledged that experiments were performed on a limited number of datasets, with plans to extend to larger, real-world workloads in future work. This raises concerns about the generalizability of the results to dynamic, production-scale cloud deployments. Second, the study does not address task dependencies or priority-based scheduling beyond job length, which are common requirements in practical cloud workflows. Third, while HABC demonstrates strong makespan and load balancing performance, the study does not evaluate critical QoS parameters such as resource reliability, fault tolerance, energy consumption, or cost efficiency, which are essential metrics for comprehensive cloud scheduler assessment. Finally, the algorithm's performance is highly sensitive to parameter configuration, particularly the number of scout, employed, and onlooker bees, and the study does not propose an adaptive tuning mechanism, limiting its robustness across varying workload characteristics.

Sefati et al. [23] proposed a QoS-aware load balancing method for cloud computing environments utilizing the Grey Wolf Optimization (GWO) algorithm, inspired by the social hierarchy and cooperative hunting behavior of grey wolves. The method operates through a structured multi-step process: first, a cluster head is selected based on the node with the highest number of neighboring VMs; the GWO algorithm then dispatches wolf agents to explore the search space, calculating a load threshold for each VM defined as $\psi = AL + \sigma$, where AL represents the average load across all VMs in the data center and σ is the standard deviation of VM loads. VMs exceeding this threshold are classified as overloaded and excluded from task assignment, while underloaded VMs are ranked using a fitness factor that combines QoS reliability scores with load balancing values. The alpha and beta wolves then guide the selection of the most suitable underloaded VM for task allocation. The reliability of each VM is quantified using an exponential decay model that accounts for failure rate and task weight, directly integrating resource dependability into the scheduling decision. The algorithm was evaluated using CloudSim across task loads ranging from 20 to 80 tasks and benchmarked against ABC, PSO, and GA. Results demonstrated that GWO achieved the lowest average makespan of 6.85, compared to 8.57 for PSO, 9.57 for ABC, and 11.41 for GA, while also achieving the lowest average response time of 5.18 seconds and the lowest average operational cost of 43.71, consistently outperforming all three competing algorithms across all evaluated metrics.

However, several critical limitations substantially constrain the practical applicability and generalizability of this work. First, the evaluation is conducted on an extremely limited scale, with a maximum of only 80 tasks and a single data center

Table 1: Comparison of related load balancing algorithms in cloud computing

Author(s)	Algorithm	Advantages	Limitations
Goyal and Singh [21]	ALBA	More uniform resource utilization in grid environments	Grid-specific, only compared against a no-load-balancing baseline
Kruekaew and Kimpan [22]	HABC, ABC	Minimizes makespan; improves load balance	Needs parameter tuning; high operational cost
Sefati et al. [23]	GWO	Lower response time and cost; considers reliability	Computational overhead; lacks security/scalability
Anbarkhan and Rakrouki [24]	EPSO	Reduces particle dimensionality via workflow task preprocessing, improved initialization ensures higher initial solution quality, achieves lower makespan and cost compared to PSO and HPSO	Limited to small-scale workloads (max 300 tasks), single-objective fitness function ignores load balancing, energy, and fault tolerance; narrow comparative evaluation against only two baselines
Lilhore et al. [25]	Hybrid WWO-ACO	Reduced task scheduling time, lower energy consumption, improved resource utilization, and superior load balancing across homogeneous and heterogeneous cloud environments	High computational overhead, performance variability across different cloud infrastructures, limited simulation scale, assumes independent tasks with no dependencies or deadlines

configuration comprising three hosts, which is far too small to draw meaningful conclusions about algorithm behavior in realistic cloud deployments that routinely handle thousands of concurrent tasks across multiple data centers. The authors themselves acknowledge this limitation, explicitly suggesting in their future work that more data centers and VMs should be incorporated to improve the proposed method. Second, the comparative evaluation is restricted to ABC, PSO, and GA, all of which are relatively dated metaheuristic baselines. The study does not benchmark GWO against more recent and competitive algorithms such as hybrid metaheuristics or other nature-inspired approaches that have demonstrated superior performance in cloud scheduling literature, which limits the strength of the performance claims. Third, while the incorporation of resource reliability into the scheduling decision is a notable contribution that distinguishes this work from purely performance-oriented approaches, the study

does not evaluate fault tolerance, energy consumption, or scalability, which are critical QoS dimensions in production cloud environments. The authors acknowledge this gap directly, recommending that future work incorporate accessibility, security, and scalability alongside reliability. Fourth, the algorithm assumes that all tasks are independent, with no dependencies or priorities among them, which is a significant simplification that reduces its applicability to real-world cloud workloads where tasks frequently form dependency graphs. The authors explicitly identify dynamic load balancing for dependent tasks as a key future research direction. Fifth, the GWO parameter configuration, including a population size of 10 and a maximum of 100 iterations, is fixed and relatively small, and the study does not provide a sensitivity analysis or adaptive tuning mechanism, raising concerns about robustness across varying workload characteristics and cloud configurations.

Anbarkhan and Rakrouki [24] proposed an Enhanced Particle Swarm Optimization (EPSO) algorithm to address the limitations of the standard PSO algorithm in scheduling workflow tasks within cloud computing environments. The core methodology involves three principal enhancements to the classical PSO framework: a workflow task model preprocessing stage that classifies tasks as either computationally intensive or IO-intensive using a directed acyclic graph (DAG) representation and merges related subtasks into unified independent task units, thereby reducing particle dimensionality, an improved particle initialization strategy that normalizes both resource computing capabilities and task data volumes, deterministically assigning the top 25% most data intensive tasks to the highest performing resources while randomly allocating the remaining 75%, ensuring a balance between initial solution quality and search space diversity and a self-adaptive fitness function defined as the reciprocal of the makespan, which aligns the particle optimization direction with the objective of minimizing overall task completion time. The algorithm was evaluated using CloudSim with an HP ProLiant DL380 G9 server configuration across workloads ranging from 10 to 300 tasks, with each algorithm executed 20 times and average results reported. Benchmarked against the standard PSO and the heuristic based HPSO, the EPSO achieved a workflow task scheduling time of 255.145 seconds for 300 tasks, compared to 394.156 seconds for HPSO and 459.562 seconds for PSO, and a corresponding cost of $401.5 versus $459.8 for HPSO and $558.3 for PSO. For independent task scheduling under the same workload, EPSO recorded 225.523 seconds against 254.246 seconds for HPSO and 327.789 seconds for PSO. Convergence analysis further demonstrated that the algorithm stabilizes after approximately 25 iterations for a 100 task workflow, reflecting the effectiveness of the dimensionality reduction strategy. However, several limitations merit critical consideration. First, the experimental evaluation is confined to a single fixed physical node configuration and a maximum of 300 tasks, which does not adequately represent the scale or heterogeneity of real world cloud deployments where thousands of concurrent tasks and heterogeneous multi datacenter environments are commonplace. Second, the particle initialization heuristic, while improving initial solution quality, applies a fixed 25/75 split between deterministic and random assignment without any adaptive mechanism to adjust this ratio based on workload characteristics or resource availability, which may lead to suboptimal initialization under highly skewed or dynamic task distributions. Third, the adaptive

function is defined solely as the reciprocal of the makespan, meaning the optimization objective is restricted to execution time minimization, important practical objectives such as load balancing, energy consumption, fault tolerance, and VM utilization are entirely absent from the fitness formulation, as the authors themselves acknowledge in their future work discussion. Fourth, while the workflow task model preprocessing introduces a meaningful reduction in problem complexity, the task merging strategy is based on a binary compute intensive versus IO-intensive classification determined by a single global threshold condition, which may oversimplify the heterogeneous dependency structures found in real scientific workflows such as those modeled by Pegasus or Montage, where tasks exhibit mixed resource demands across different pipeline stages. Fifth, the comparative evaluation is limited to two baselines the standard PSO and the HPSO of and does not include comparisons against other contemporary metaheuristic approaches such as ACO based, GA based, or more recent hybrid methods, which significantly constrains the ability to assess the EPSO algorithm's relative competitiveness within the broader state of the art.

Lilhore et al. [25] proposed a hybrid multi-objective optimization algorithm that integrates Water Wave Optimization (WWO) and ACO to address load balancing and resource allocation challenges in cloud computing environments. The hybrid model leverages ACO's local search capabilities for task scheduling, wherein ants probabilistically select resources based on pheromone levels and heuristic information, while WWO's global exploration mechanism, modeled on wave propagation, refraction, and breaking phenomena, is employed for broader resource allocation optimization. The two algorithms operate sequentially, with ACO generating initial solutions that serve as the starting population for the WWO phase, and the best solutions from both phases are combined through a Pareto-dominance framework to simultaneously optimize four competing objectives: response time, resource consumption, operational cost, and energy consumption. The algorithm was evaluated using CloudSim Plus 6.2.2 across five experimental scenarios, with workloads ranging from 500 to 2500 tasks and VM configurations of 50 to 200, and benchmarked against GA, SMO, WWO, and ACO. Results demonstrated that WWO-ACO achieved an average task scheduling duration of 1107.8 seconds for 2000 tasks and 150 VMs, substantially outperforming ACO (1407.32 s), GA (7378.72 s), SMO (7578.32 s), and WWO (8587.7 s). Additionally, the algorithm reduced energy consumption to 335.0 kJ for 100 VMs and 2500 tasks, compared to 400.4 kJ for GA and 344.9 kJ for ACO, while achieving balance degree values ranging from 0.87 to 0.95 across 200 VM configurations, indicating consistently equitable workload distribution. However, several limitations warrant critical consideration. First, the authors themselves acknowledge that the algorithm incurs significant computational overhead, with a time complexity of $O((N_{ants} \times N_{iter} \times D) + (N_{waves} \times N_{iter} \times D))$, which may become prohibitive in large-scale or highly dynamic cloud deployments where real-time scheduling decisions are required. Second, the paper acknowledges performance variability across different cloud infrastructures and workload distributions, with the authors noting that additional parameter tuning may be necessary for different deployment contexts, which limits the algorithm's out-of-the-box generalizability. Third, while the evaluation employs both synthetic workloads and a real-world HPC2N log trace, the simulation environment is confined

to a fixed configuration of three data centers and ten hosts, which does not fully represent the scale and heterogeneity of production cloud deployments. Fourth, the study does not address task dependencies, deadlines, or priority constraints, assuming all tasks to be independent and free, which is a significant restriction given that real-world cloud workflows frequently involve complex inter-task relationships. Finally, while the paper compares the algorithm against GA, SMO, WWO, and ACO, it does not benchmark against other recent hybrid metaheuristics specifically designed for cloud load balancing, such as GWO based or PSO based hybrid approaches, which limits the comprehensiveness of the comparative evaluation.

3 Proposed Model

In cloud computing, vast networks of data centers and users are spread globally. When numerous user requests flood the cloud simultaneously, efficient organization and service delivery are essential. The cloud must distribute workloads evenly across resources to ensure fairness. Load balancing remains a key challenge, requiring equal distribution of tasks to maintain user satisfaction. Various load-balancing algorithms exist for distributed, grid, and cloud systems [10]. While metaheuristic approaches such as GA, GWO, PSO, and WOA have been applied to cloud scheduling, RDA a nature-inspired metaheuristic modeled on the mating and territorial behavior of Scottish Red Deerchas not previously been investigated for cloud load balancing. RDA's biologically inspired exploration exploitation mechanism, which combines competitive fighting, harem formation, and multi-level mating strategies, offers a structurally distinct search dynamic compared to existing metaheuristic schedulers. This structural novelty motivates its application to the cloud scheduling problem, where balancing multiple competing QoS objectives under heterogeneous resource conditions remains an open challenge. This study therefore proposes the first RDA based load balancing approach for cloud environments, incorporating a multi-objective QoS aware fitness function that simultaneously optimizes response time, execution cost, resource utilization, and workload distribution during task scheduling.

3.1 Problem Statement

Consider cloud C, which consists of N physical machines, each containing M VMs.

$$C = \{PM_1, PM_2, \ldots, PM_N\} \tag{1}$$

Additionally, every physical machine consists of multiple VMs configured as follows:

$$PM_i = \{VM_{i1}, VM_{i2}, \ldots, VM_{iM}\} \tag{2}$$

VM_1 represents the first virtual machine, and VM_M denotes the final one. Our aim is to minimize costs and maximize the utilization of resources by evenly distributing workload across all the VMs. Efficient load balancing is the key to our approach; otherwise, the system would waste unnecessary time and energy on its tasks. To tackle

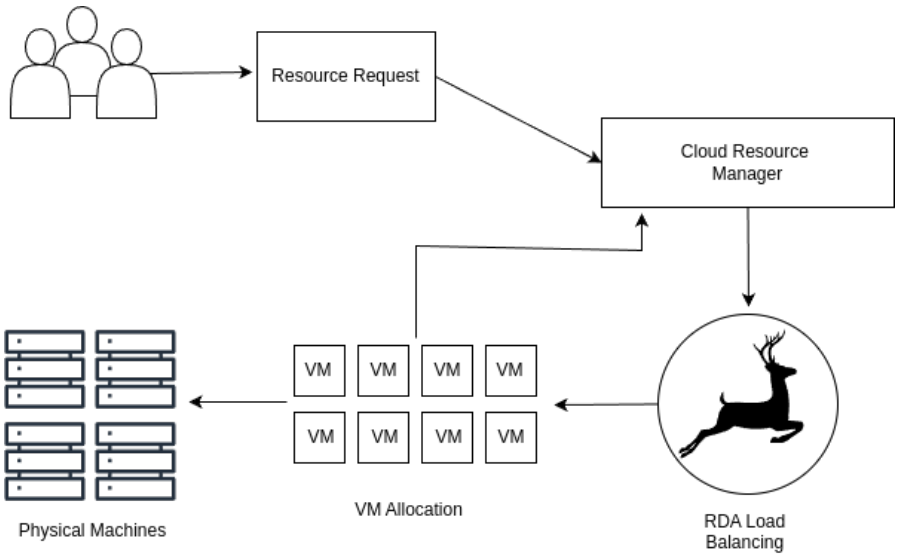


Fig. 2: The proposed load-balancing system

this, our study proposes an efficient, multi-objective load-balancing approach using the RDA algorithm.

Our model treats the resource allocation as a multi-objective optimization problem and seeks to:

1. Minimize the makespan time to enhance task completion time and efficiency,
2. Maximize utilization of resources to improve infrastructure efficiency,
3. Minimize costs to make it economically viable,
4. Reduce response time to maintain QoS.

These objectives tend to be conflicting with one another, and they need to be balanced properly. For example, aggressively reducing makespan may raise the demand for provisioning more resources, which can raise costs and lower overall utilization [23]. Hence,The model employs a weighted approach towards Red Deer and these objectives, allowing system administrators to assign weights based on their intended use of the VMs. This helps in maintaining the overall system perform under evolving requirements.

3.2 Parameters of QoS

Makespan, utilization of resources, response time, and cost-effectiveness are performance measures that, collectively, constitute a holistic framework for quantifying the effectiveness of resource allocation policies [22]. The model introduced here attempts to promote resource allocation in cloud environments by balancing these performance measures.

The proposed model considers heterogeneous virtual machines with varying computational capacities, execution costs, and availability levels in order to better simulate realistic cloud computing environments.

3.2.1 Makespan

Makespan time represents the total time required to complete the scheduled tasks [23]. In our model, we are considering the maximum makespan time among all VMs, it is given as following:

$$\text{Makespan} = \max\{CT_1, CT_2, \ldots, CT_M\} \tag{3}$$

Where $\mathrm{CT_i}$ represents the completion time of ith virtual machine, and M is the total number of available VMs. Our goal is to reduce makespan time to guarantee timely execution of tasks.

3.2.2 Resource Utilization

Resource utilization is the ratio of time a VM is actively processing tasks to the makespan time [23]. Resource utilization for an individual VM:

$$\text{Utilization}_{VM_j} = \frac{\sum_{i=1}^{n} PT_{ij}}{\text{Makespan}} \tag{4}$$

Where PT_{ij} represents the processing time of the ith task which was processed on $j^{th}VM$. The total resource utilization across the cloud is then determined by:

$$\text{Resource Utilization} = \frac{\sum_{j=1}^{m} \text{Utilization}_{VM_j}}{m} \tag{5}$$

This metric provides information on how effectively available computational resources are being used in the present situation. Increased utilization ratios indicate better resource management, which can reduce the operational expenses as well as the environmental impact.

3.2.3 Response Time (RT)

We model the response time in our model by utilizing SpaceShared basis queuing behavior of tasks on Vms. SpaceShared is a resource allocation policy where each cloudlet uses a single core at a time, and the remaining wait in a queue until other resources become available to be utilized [26].

The response time is defined as the time that a cloudlet takes from the time of arrival to its completion time.

The execution time for cloudlet C_i on VM v_j is:

$$RT_{exec}(C_i, v_j) = \frac{Len}{MIPS_j} \tag{6}$$

Where Len is the cloudlet length in Million Instructions (MI), and $MIPS_j$ is the processing speed of VM v_j.

Since the SpaceShared policy allows limited concurrent execution on a VM, cloudlets assigned to the same VM may experience queuing delays before execution. Therefore, the response time $RT(C_i)$ includes both waiting time and execution time during task scheduling. The average response time is:

$$RT_{avg} = \frac{1}{n}\sum_{i=1}^{n}(FT_i - AT_i) \tag{7}$$

where FT_i and AT_i represent the finish time and arrival time of cloudlet i, respectively.

Lower values indicate better scheduling and faster execution.

3.2.4 Cost

The cost model incorporates both resource consumption and temporal factors [23]. For a set of k VMs assigned to user requests, the total cost is expressed as:

$$\text{Cost} = \sum_{i=1}^{k} C_i \cdot T_i \tag{8}$$

where C_i represents the execution cost rate of VM_i, and T_i denotes the total execution time of tasks assigned to that VM. The overall cost increases with longer execution durations and higher resource consumption. Minimizing this parameter helps improve the economic efficiency of cloud resource allocation.

3.3 Fitness Function

All the QoS parameters use different units, Some parameters (e.g., utilization) have a positive contribution to fitness, while others (e.g., response time, cost, makespan) have a negative impact [23]. Hence, they need to be normalised on a single scale to measure the objective function.

Equation 9 and 10 represents the normalization formula for the minimizing and maximizing parameters, respectively [27].

$$N_{Q^i} = \begin{cases} \frac{Q^i - Q^i_{\min}}{Q^i_{\max} - Q^i_{\min}}, & Q^i_{\max} \neq Q^i_{\min} \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

$$N_{Q^i} = \begin{cases} \frac{Q^i_{\max} - Q^i}{Q^i_{\max} - Q^i_{\min}}, & Q^i_{\max} \neq Q^i_{\min} \\ 1, & \text{otherwise} \end{cases} \tag{10}$$

For minimization objectives such as makespan, response time, and execution cost, lower normalized values indicate better scheduling quality. In contrast, for maximization objectives such as resource utilization, higher normalized values represent better performance.

The fitness function is given as Algorithm 1.

To effectively balance the QoS parameters during resource allocation, a unified fitness function is designed which evaluates each candidate solution based on normalized QoS metrics, ensuring correctness regardless of their original scales [23].

Algorithm 1 Fitness Function

```
procedure EvaluateFitness(solution)
    Initialize cost, responseTime, and VM processing statistics
    Retrieve VM parameters including MIPS, execution cost, and availability
    for each cloudlet_i in solution do
        Decode solution_i to obtain assigned VM_j
        Calculate execution time of cloudlet_i on VM_j
        Update VM processing time and scheduling metrics
    end for
    Calculate makespan Eq. 3, avgUtilization Eq. 5, avgResponseTime Eq. 7, totalCost Eq. 8
    Normalize all QoS metrics
    Compute final fitness value using the weighted sum model
end procedure
```

The overall fitness F of a solution is computed as:

$$Fitness = w_1 \cdot N_{makespan} + w_2 \cdot N_{utilization} + w_3 \cdot N_{cost} + w_4 \cdot N_{RT} \tag{11}$$

where $N_{makespan}$ 3, $N_{utilization}$ 4, 5, N_{RT} 6, 7, and N_{cost} 8 are the normalized parameters, and w_1 to w_4 are user-defined weights satisfying $\sum w_i = 1$. Optimizing solely one parameter could lead to resource under utilization. Hence, By assigning appropriate weights, users could emphasize on specific objectives based on their operational context.

In the proposed model, each candidate solution represents a cloudlet-to-VM mapping vector, where each index corresponds to a cloudlet and the stored value indicates the VM assigned for its execution. The scheduling objective is to identify an optimal mapping that balances workload distribution while satisfying multiple QoS requirements.

The optimization process terminates when either the maximum number of iterations is reached or the improvement in fitness value becomes negligible across consecutive iterations.

4 Development of the RDA According to the Proposed Model

In the proposed approach, the primary objective is to support real-time cloud applications, where adherence to task deadlines is critical. Each task is associated with a user-defined execution time constraint, and the system endeavors to complete the execution of the task within the specified timeframe to uphold QoS requirements.

The RDA is a nature-inspired metaheuristic technique modeled after the mating behavior of Scottish Red Deer during the breeding season. In the context of cloud computing, this algorithm is adapted to solve the resource allocation problem, specifically mapping cloudlets to VMs by simulating the social dynamics of Red Deer. Each Red Deer represents a candidate solution encoding a potential mapping of cloudlets to VMs.

Algorithm 2 RDA for Load Balancing in Cloud Computing

```
procedure RedDeerAlgorithm(numVMs, numCloudlets)
    Initialize: populationSize, numMales, numHinds, α, β, γ, UB,
                LB, maxIterations
    numCommanders ← ⌊γ × numMales⌋
    numStags ← numMales − numCommanders
    bestFitness ← ∞
    Initialize Population: Generate populationSize random cloudlet-to-VM mapping solutions
    for all deer d in population do
        d.fitness ← 1 EvaluateFitness(d.position)
    end for
    Sort population in ascending order of fitness
    males ← top numMales individuals
    hinds ← remaining individuals
    commanders ← top numCommanders from males
    stags ← remaining males
    for iter = 1 to maxIterations do                                  ▷ Roaring Phase
        for all male in males do
            Update male position if fitness improves via roaring
        end for
        Sort males by fitness
        Update commanders and stags
                                                                      ▷ Fighting Phase
        for all commander in commanders do
            Select random stag
            Generate new positions by fighting
            Update commander with best position
        end for
                                                                      ▷ Harem Formation
        Calculate relative fitness of commanders for harem allocation
        Divide hinds into harems proportional to fitness
                                                                      ▷ Mating Phase
        for all commander in commanders do
            Mate with a proportion α% of hinds in the same harem
            Mate with a proportion β% of hinds from another harem
        end for
        for all stag in stags do
            Find nearest hind and mate
        end for
                                                                      ▷ Selection Phase
        Select the best individuals for the next population using tournament selection
        Update bestPosition and bestFitness if improved
        Break if convergence criteria are satisfied
    end for
    return Mapping of cloudlets to VMs using bestPosition
end procedure
```

The algorithm outputs the best mapping of cloudlets to VMs that balances makespan, resource utilization, cost efficiency, and response time.

4.1 Generation of Initial Red Deer Population

A population of 100 Red Deer is initialized, where each Red Deer represents a possible cloudlet-to-VM mapping solution, as illustrated in Figure 3. Each Red Deer is encoded as a one-dimensional array of size equal to the number of cloudlets, where each element stores the identifier of the VM assigned to the corresponding cloudlet. For instance, a Red Deer looks like:

$$RD_i = [x_1, x_2, x_3, \ldots, x_{N_c}] \tag{12}$$

Figure 3 illustrates the mapping representation used in the proposed model. Each row corresponds to one Red Deer (candidate scheduling solution), while each column represents a cloudlet. The value stored at each position denotes the VM identifier assigned to the corresponding cloudlet. For example, if the value at the third index is VM_2, then the third cloudlet is allocated to virtual machine 2 for execution.

After generating the population, the fitness of each Red Deer is evaluated using the proposed objective function Algo 1 based on makespan, resource utilization, response time, and execution cost.

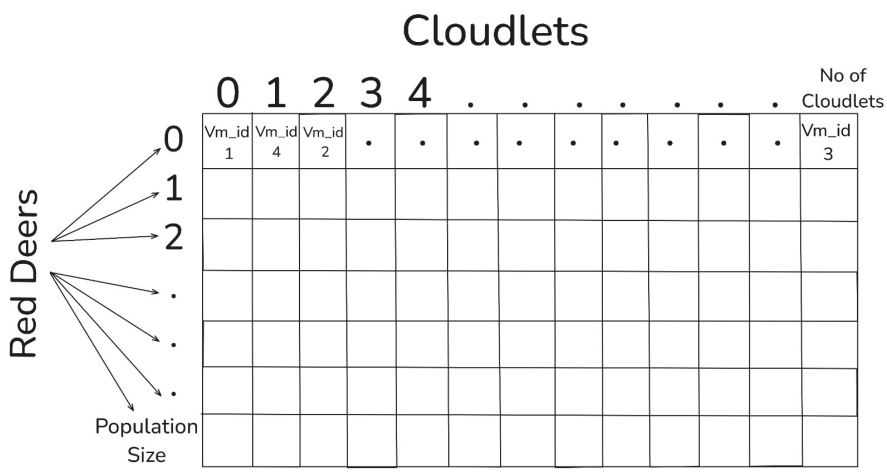


Fig. 3: Representation of a Red Deer as a cloudlet-to-VM mapping solution

Once fitness values are obtained, the population is sorted in ascending order of fitness (lower is better), and the top-performing N_{male} (a predefined algorithm parameter) Red Deer are selected as males from which top $\gamma\%$ of the males are chosen as commanders, while the remaining male Red Deer are classified as stags and the rest of the population forms the group of hinds.

Top N_{male} Red Deer are considered as males:

$$M = \{RD_1, RD_2, \ldots, RD_{N_{\text{male}}}\} \tag{13}$$

The remaining individuals are considered as hinds:

$$H = \{RD_{N_{\text{male}}+1}, RD_{N_{\text{male}}+2}, \ldots, RD_{N_{\text{pop}}}\} \tag{14}$$

Here, N_{male} controls the intensification capability of the algorithm by preserving high-quality solutions, while the hind population contributes to diversification and exploration of the search space.

4.2 Roaring Process

In the roaring phase, each male Red Deer attempts to improve its current solution (i.e., cloudlet-to-VM mapping) by exploring its neighborhood in the solution space. This behavior mimics the natural tendency of Red Deer to expand their territory and display dominance by roaring.

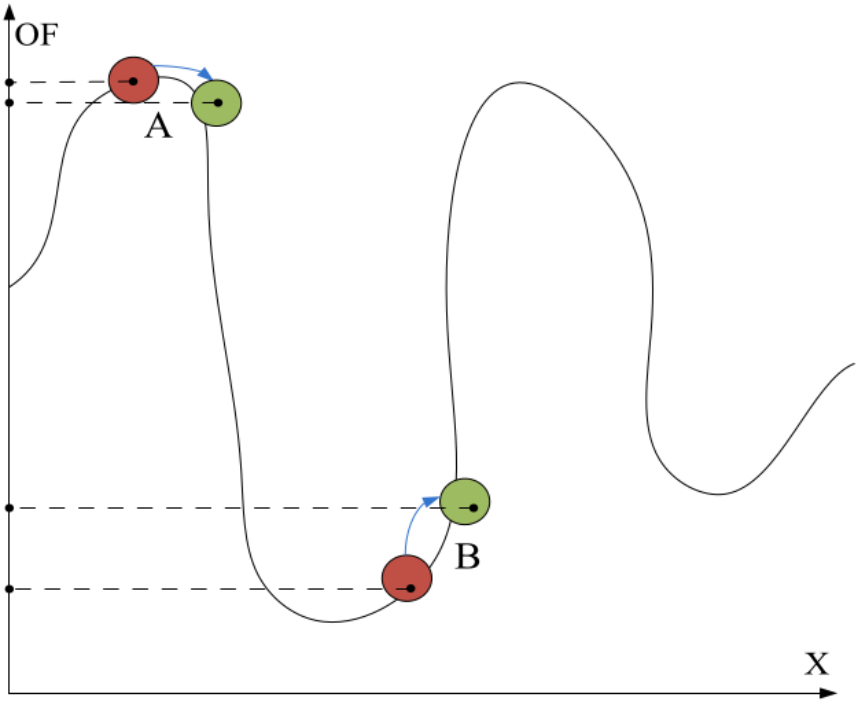


Fig. 4: Roaring Process of male Red Deer [18]

To simulate this behavior computationally, each male Red Deer $RD_i \in M$ updates its position according to the following rule [18]:

$$male_i^{\text{new}} = \begin{cases} male_i^{\text{old}} + a_1 \cdot (((UB - LB) \cdot a_2) + LB), & \text{if } a_3 \geq 0.5 \\ male_i^{\text{old}} - a_1 \cdot (((UB - LB) \cdot a_2) + LB), & \text{if } a_3 < 0.5 \end{cases} \tag{15}$$

Where:

- $male_i^{\text{old}}$ is the current Red Deer position (i.e., current cloudlet-to-VM assignment),
- $male_i^{\text{new}}$ is the updated solution after roaring,
- $UB = N_{VM}$ and $LB = 0$, which denotes the upper and lower bounds of the solution space, respectively. All the numbers in this range represent unique `vm_ids`,
- $a_1, a_2, a_3 \in [0, 1]$ are uniformly distributed random numbers.

Each male RD generates a new position using the above update rule. The updated solution is accepted only if it yields a better fitness value compared to the previous one. This ensures that the exploration during roaring leads to quality improvements in the mapping.

Figure 4 illustrates two scenarios chosen from the solution space to show the effect of roaring in isolation. Observe that cases A and B normally happen within the roaring period. In case A, the new position is accepted because its objective function value (Eq. 11) is lower than that of the old position. In case B, the new solution is not acceptable. Observe again that the y-axis is the Objective function value, and the x-axis is the positions of males in this figure.

4.3 Selection of the best male Red Deer as commanders

In nature, not all male Red Deer possess the same level of strength, dominance, or attractiveness. Some exhibit superior traits and are more successful in claiming and defending territories. This variation is modeled in the RDA by classifying the male Red Deer into two distinct groups: *commanders* and *stags* [18].

In our cloudlet-to-VM mapping scenario, each male RD represents a potentially optimal mapping configuration. To further intensify the search, we designate the top $\gamma\%$ of male Red Deer (based on fitness value) as commanders and the remaining ones as stags.

The number of commanders N_{Com} is computed using the following formula:

$$N_{\text{Com}} = \text{round}(\gamma \cdot N_{\text{male}}) \tag{16}$$

where:

- $\gamma \in [0, 1]$ is a user-defined parameter representing the percentage of elite males to be chosen as commanders,
- N_{male} is the total number of male Red Deer.

After selecting the commanders, the remaining male Red Deer are assigned the role of stags. The number of stags N_{stag} is calculated as:

$$N_{\text{stag}} = N_{\text{male}} - N_{\text{Com}} \tag{17}$$

This division of roles creates a hierarchy among the male Red Deer, which is later used in the fighting and mating phases [18]. In our implementation, we set $\gamma =$

0.4, meaning 40% of male Red Deer are chosen as commanders which was giving experimentally best outcome.

4.4 Fight between male commanders and stags

Once the male Red Deer have been divided into commanders and stags, the next step is to simulate the natural behavior of dominance through fights. Each commander is randomly paired with a stag to engage in a competition to improve the population's overall fitness [18]. These confrontations result in the generation of new candidate solutions through exploitation of the solution space.

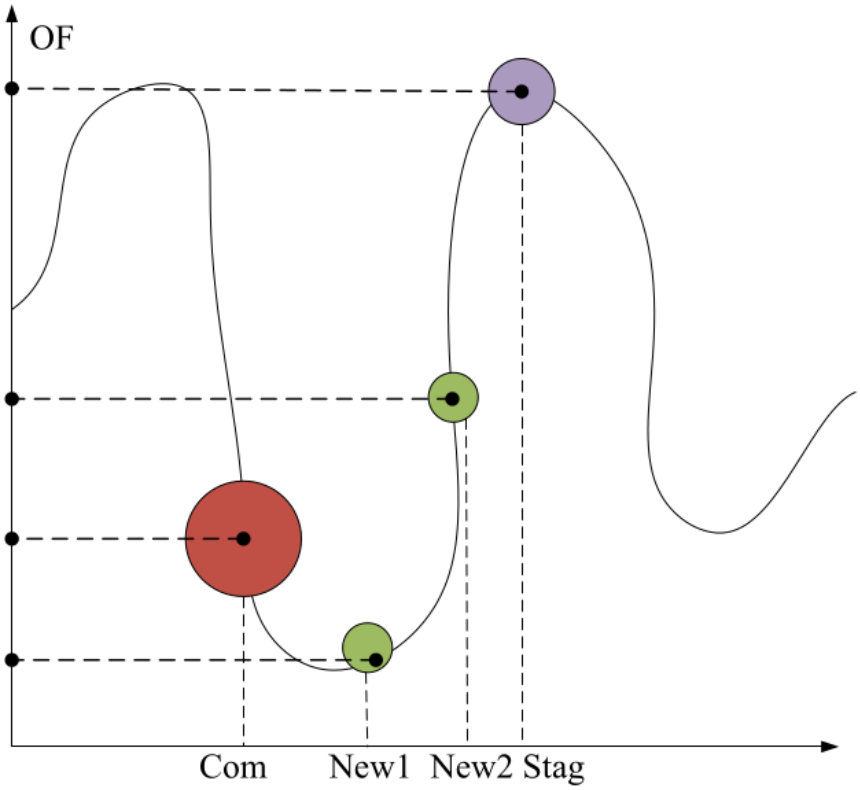


Fig. 5: Fight between a commander and a stag [18]

For each such pairing, two new solutions are generated using the following equations:

$$\text{New}_1 = \frac{(\text{Com} + \text{Stag})}{2} + b_1 \cdot ((\text{UB} - \text{LB}) \cdot b_2 + \text{LB}) \tag{18}$$

$$\text{New}_2 = \frac{(\text{Com} + \text{Stag})}{2} - b_1 \cdot ((\text{UB} - \text{LB}) \cdot b_2 + \text{LB}) \tag{19}$$

where:

- Com and Stag denotes the current position vectors (solutions - Cloudlet to VMs mapping) of the commander and stag, respectively,
- b_1 and b_2 are random variables drawn from a uniform distribution over the interval [0, 1].

The commander is replaced with best solution after evaluating their objective function (eq.11) out of all the four possible solution (commander, stag, and the two new offspring 18) [18].

4.5 Form harems

In nature, each male commander claims a group of hinds, forming a harem based on his strength and dominance. In our solution, this behavior is simulated by associating a portion of the hinds population with each commander based on their fitness values. A lower fitness value indicates a better scheduling solution and therefore results in a larger harem allocation.

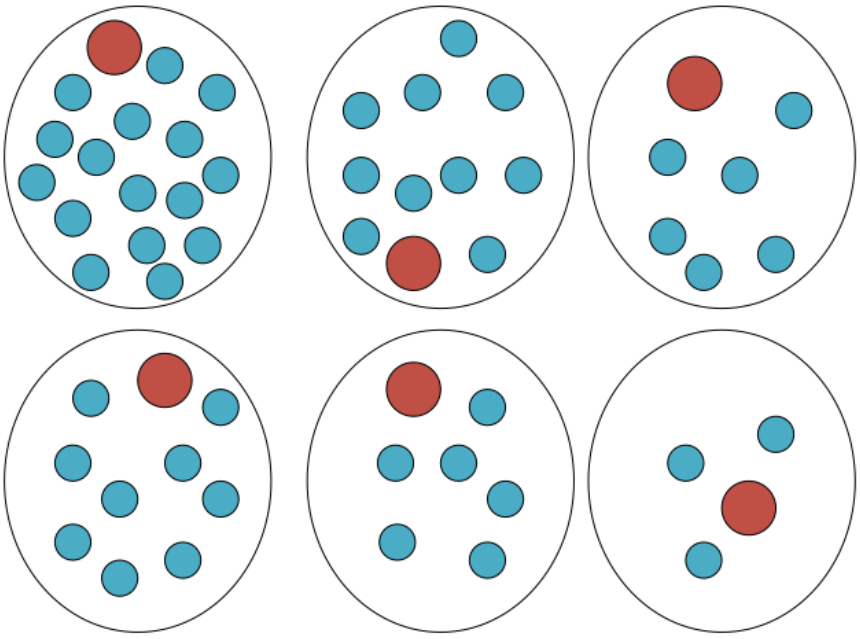


Fig. 6: Harems of each commander [18]

Calculate the relative power V_n of each commander using the following equation:

$$V_n = v_n - \max\{v_i\} \tag{20}$$

where:

- v_n is the fitness value (objective function value) of the n-th commander,
- $\max\{v_i\}$ represents the worst (here, maximum) fitness among all commanders, ensuring normalization by fitness proximity to optimality.

Then, we normalize this value to determine the portion of hinds to be assigned to each commander:

$$P_n = \frac{V_n}{\sum_{i=1}^{N_{Com}} V_i} \tag{21}$$

where:

- P_n is the normalized power of the n-th commander,
- N_{Com} denotes the total number of commanders.

Now, based on this proportional power, the number of hinds allocated to the n-th commander's harem is computed as:

$$N.\text{harem}_n = \text{round}(P_n \cdot N_{\text{hind}}) \tag{22}$$

where:

- $N.\text{harem}_n$ is the number of hinds of the n-th commander's harem,
- N_{hind} is the total number of hinds in the population.

Hinds are then randomly assigned to commanders based on the computed values $N.\text{harem}_n$. Each selected group of hinds, together with its corresponding commander, constitutes one harem.

4.6 Mating of commanders with hinds in his harem

Like other species in nature, Red Deer mates with each other. Here commander mates with α% of hinds of his harem and generates offsprings. It is calculated as:

$$N.\text{Harem}_n^{mate} = \text{round}(\alpha \cdot N.\text{Harem}_n) \tag{23}$$

where:

- $N.\text{Harem}_n^{mate}$ is the number of hinds from the n-th harem that will mate with the commander,
- $N.\text{Harem}_n$ is the total number of hinds in that harem,
- α [0-1] is a user-defined uniform model parameter that controls the mating percentage.

Once the hinds for mating are selected randomly from each harem, a new offspring (solution) is produced using the following equation:

$$\text{offs} = \frac{\text{Com} + \text{Hind}}{2} + c \cdot (UB - LB) \tag{24}$$

where:

- Com and Hind represent the current position vectors (solutions - Cloudlet to VMs mapping) of the commander and a selected hind,
- offs is the new offspring (i.e., the new candidate solution),
- c is a random number generated from a uniform distribution over the interval [0, 1].

4.7 Mating of commander with hinds of another harem

Allow each commander to mate with β% of hinds from another randomly selected harem [18].

Let i be the index of a randomly selected harem which is not itself. The number of hinds in the i-th harem that mate with the current commander is given by:

$$N.\text{Harem}_i^{mate} = \text{round}(\beta \cdot N.\text{Harem}_i) \tag{25}$$

where:

- $N.\text{Harem}_i^{mate}$ is the number of hinds from the i-th harem that will mate with the commander,
- $N.\text{Harem}_i$ is the number of hinds in the i-th harem,
- β is a user-defined parameter controlling the proportion of hinds involved in inter-harem mating, range value of this parameter is between zero and one.

Each of the $N.\text{Harem}_i^{mate}$ hinds selected from the i-th harem mates with the current commander using the same offspring generation formula defined in 4.6.

4.8 Mating of stag with the nearest hind

Unlike commanders, stags do not form harems but still participate in the mating phase. Each stag mates with the nearest hind in the solution space in order to generate locally improved offspring solutions.

The nearest hind is determined using Euclidean distance in the J-dimensional solution space. The distance between a stag and the i-th hind is calculated as:

$$d_i = \sqrt{\sum_{j=1}^{n} (\text{stag}_j - \text{hind}_{i,j})^2} \tag{26}$$

where:

- d_i is the distance between the stag and the i-th hind,
- stag_j and $\text{hind}_{i,j}$ represent the j-th dimension of the stag and of the i-th hind, respectively,

After identifying the nearest hind (i.e., the one with the smallest d_i), the stag mates with it using the same equation as 4.6, replacing the commander's role with that of the stag.

4.9 Select the next generation

The next generation is formed by combining the current solutions and newly generated offspring.

To achieve this, two selection strategies are used:

1. **Elite Preservation:** High-quality male red deer solutions, including commanders and stags, are preserved to maintain elite scheduling solutions across generations.
2. **Offspring Selection:** The remainder of the population is filled by selecting individuals from the pool of all hinds and the offspring generated through the mating processes. This selection is performed using the tournament selection method.

4.10 Stopping condition

The optimization process continues until the maximum number of iterations is reached or convergence is observed in the fitness values across consecutive iterations. Based on preliminary experimentation and parameter sensitivity analysis, the iteration count was set to 100 for the reported simulations.

Figure 1 and Algorithm 2 provides a summarized overview of the main steps involved in the RDA. It highlights the initialization, male classification, roaring, fighting, harem formation, mating, and selection processes that collectively guide the search for optimal load balancing in the cloud environment.

5 Experimental Setup

To evaluate the performance of the proposed scheduling approach under controlled and repeatable conditions, CloudSim Plus was used to simulate a virtual cloud computing environment. CloudSim Plus is an open and extensible simulation framework via which cloud computing infrastructures and resource provisioning policies can be simulated and modeled. It was created at the CLOUDS Laboratory, Department of Computer and Software Engineering, University of Melbourne [28], it provides a controlled environment to conduct experiments without the need for a real cloud infrastructure. It allowed us to compare the performance of our proposed algorithm with other existing algorithms effectively. Moreover, CloudSim Plus supports customization by enabling developers to extend or replace its core classes, making it suitable for implementing new scheduling or provisioning policies [29].

5.1 Simulation data and parameters

The virtual machines used during simulation were instantiated using the VM configurations listed in Table 3, depending on the workload and scalability scenario under evaluation.

To simulate heterogeneous cloud environments, multiple host configurations with varying computational capacities were used, as summarized in Table 2. The initial experiments were conducted using a smaller baseline setup, while larger-scale configurations containing up to 20 hosts were later evaluated to analyze scalability and scheduling behavior under increasing infrastructure complexity. The hosts differed in

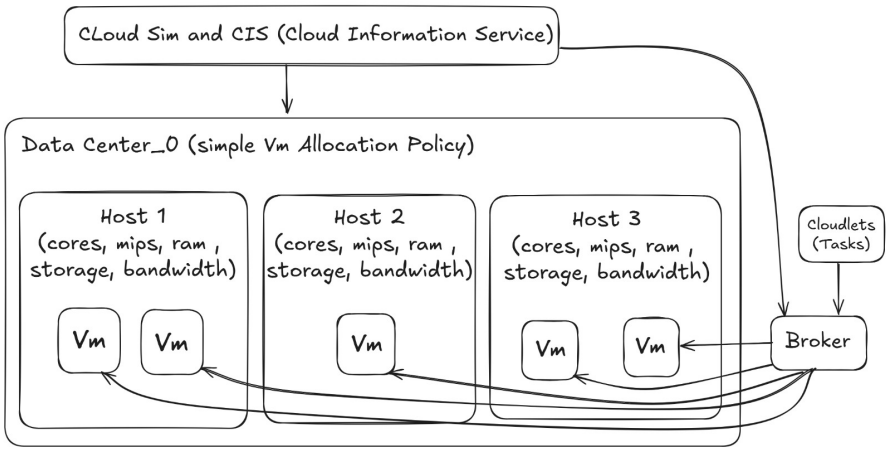


Fig. 7: Basic architecture of CloudSim Plus

Table 2: Types of hosts used in Simulation

Host ID	Cores	Processing speed, MIPS	RAM, MB	Storage, MB	Bandwidth, Mbps
1	4	5000	204,800	1,048,576	102,400
2	2	2500	102,400	1,048,576	102,400
3	1	1000	51,200	1,048,576	102,400

Table 3: Virtual Machine Types Used in Simulation

VM Type	MIPS	RAM (MB)	Cost Rate	Availability
VM1	1000	2048	0.05	1.0
VM2	2000	4096	0.08	0.9
VM3	3000	6144	0.10	0.8
VM4	4000	8192	0.15	0.7

terms of processing speed, number of cores, memory capacity, and other hardware characteristics in order to better represent realistic cloud computing environments.

In addition to the initial small-scale setup, larger-scale experiments were also conducted to evaluate the scalability of the proposed model. The extended simulation environment included up to 20 heterogeneous hosts, 50 virtual machines, and workload sizes ranging from 100 to 2000 cloudlets. These configurations were used to analyze the behavior of the proposed approach under increasing workload and infrastructure complexity.

The RDA parameter configuration shown in Table 6 was selected based on preliminary experimentation and parameter sensitivity analysis. Various combinations of

Table 4: Cloudlet Workload Configuration

Parameter	Value
Cloudlet Length	1000 – 20000 MI
Number of Cloudlets	100 – 2000
Cloudlet Scheduling Policy	SpaceShared
Utilization Model	Full Utilization

Table 5: Simulation Scale Configuration

Parameter	Value
Number of Hosts	3 – 20
Number of Virtual Machines	10 – 50
Number of Cloudlets	100 – 2000
Host Core Configuration	4 – 16 cores
Host RAM Configuration	16 GB – 64 GB
VM Scheduling Policy	SpaceShared
Cloudlet Scheduling Policy	SpaceShared

Table 6: Parameters of RDA

Parameters	Value
Population size(no. of solutions)	100
Male population(no. of males)	20%
Maximum iteration	100
Alpha(% of hinds a commander mates with in his harem)	0.35
Beta(% of hinds a commander mates with in another harem)	0.3
Gamma(Fraction of males selected as commanders)	0.45
LB(Lower Bound)	0.0
UB(Upper Bound)	$N_{VM} - 1$

population size, male-to-female ratio, and mating behavior percentages (α, β, γ) were tested. The parameter configuration providing stable and competitive scheduling performance in terms of makespan, response time, and resource utilization was selected for the reported experiments. Specifically, a population size of 100 and 20% males with a commander selection ratio of 40% provided a good balance between exploration and exploitation. The bounds for solution values were defined based on task and resource limits in the simulation environment.

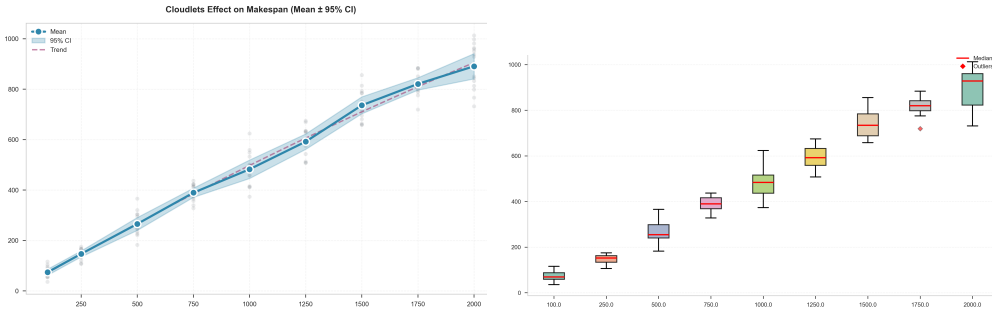
All the experiments were performed on a Lenovo Yoga 6 laptop equipped with an Intel i5 13th Generation H-series processor and 16 GB RAM.

The proposed RDA model was benchmarked against several metaheuristic scheduling algorithms including GA, PSO, GWO, and WOA using identical simulation configurations and workload conditions.

6 Results and Analysis

6.1 Makespan Analysis

Makespan is one of the most important performance metrics in cloud task scheduling, as it represents the total completion time required to execute all cloudlets. Lower makespan values indicate better workload distribution and improved resource utilization across virtual machines.



(a) Makespan under varying cloudlet workloads

(b) Boxplot analysis of makespan distribution

Fig. 8: Makespan analysis of the proposed scheduling approach under different workload configurations

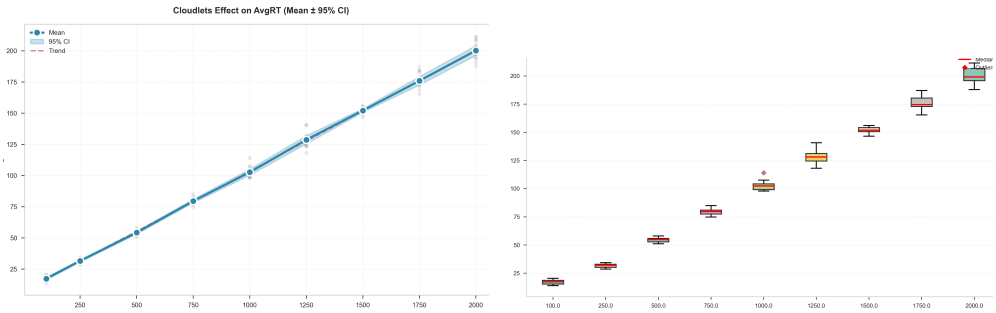
Figure 8 illustrates the variation in makespan under increasing cloudlet workloads. As the number of cloudlets increased from 100 to 2000, the makespan increased gradually due to higher scheduling complexity and resource at capacity within the cloud environment. However, the increase remained relatively stable, indicating effective workload distribution by the proposed RDA scheduler even under larger-scale workloads.

The boxplot analysis further demonstrates that the variation in makespan remained comparatively controlled across repeated simulation runs, indicating stable scheduling behavior and robustness of the proposed approach. The obtained results suggest that the RDA-based scheduler was capable of maintaining balanced virtual machine utilization while avoiding excessive growth in task completion time under heterogeneous workload conditions.

6.2 Response Time Analysis

Response time is an important QoS metric in cloud scheduling because it reflects the delay experienced before task completion. Lower response time indicates faster scheduling decisions and improved utilization of computational resources.

Figure 9 illustrates the variation in average response time under increasing cloudlet workloads. As the number of cloudlets increased from 100 to 2000, response time increased gradually due to higher scheduling complexity and increased competition for



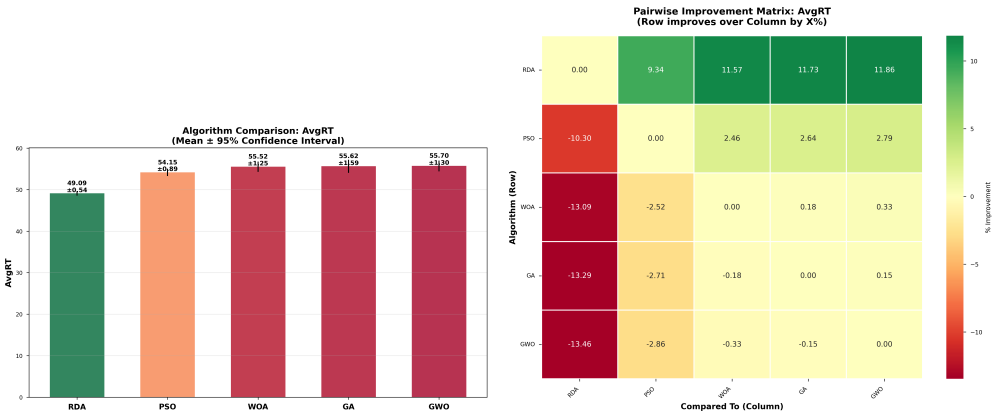
(a) Average response time under varying cloudlet workloads

(b) Boxplot analysis of response time distribution

Fig. 9: Response time analysis of RDA under varying cloudlet workloads

computational resources within the cloud environment. The statistical analysis tells that the proposed RDA model maintained relatively stable and lower mean response time growth across varying workload conditions.

The boxplot analysis also indicates comparatively stable response time distribution across repeated simulation runs, demonstrating robustness and consistency of the proposed scheduling mechanism under heterogeneous workload conditions.



(a) Algorithm-wise response time comparison

(b) Percentage improvement matrix for response time

Fig. 10: Comparative response time analysis of the evaluated scheduling algorithms

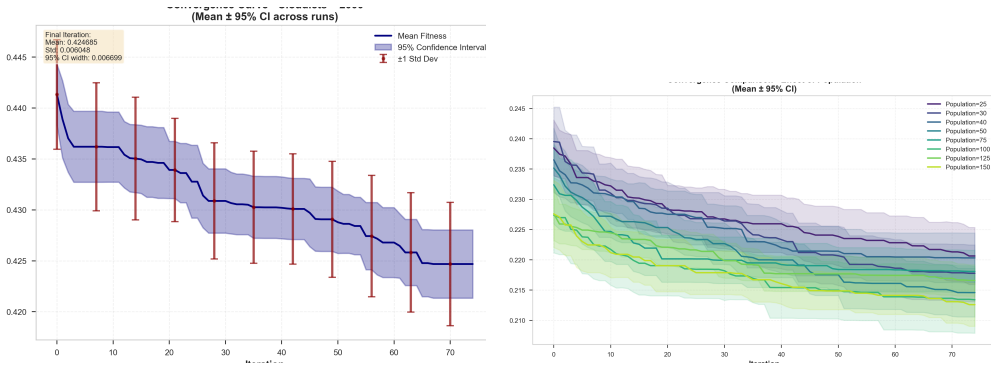
Figure 10 presents the comparative response time performance of the evaluated scheduling algorithms. The proposed RDA model achieved an average response time of 49.09 ms, which was lower than the values obtained by GA, GWO, PSO, and WOA across most workload configurations. The improvement matrix further highlights the

percentage performance gains achieved by the proposed scheduler over competing approaches.

6.3 Convergence Analysis

The convergence behavior of the proposed RDA model was analyzed to evaluate the stability and optimization capability of the algorithm during iterative scheduling.

Figure 11 illustrates the convergence characteristics of the proposed model under different workload and parameter configurations. The fitness value decreased rapidly during the initial optimization stages, indicating effective exploration of the search space. As the iterations progressed, the curves gradually converged after approximately 62 iterations, demonstrating refinement of scheduling solutions and improved exploitation behavior.



(a) Convergence behavior of RDA under 2000 cloudlet workloads

(b) Convergence behavior under different population sizes

Fig. 11: Convergence analysis of the proposed RDA model under different parameter and workload configurations

The convergence analysis under increasing cloudlet workloads show that the proposed model maintained it's optimization behavior even for larger-scale scenarios involving up to 2000 cloudlets. The final fitness value improved by approximately 40%, indicating progressive enhancement in scheduling quality during iterative optimization.

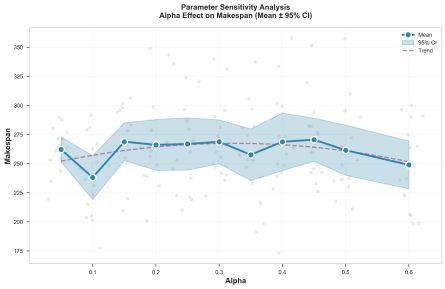
The population-based convergence analysis also indicates that smaller population sizes reduces search diversity and increases the possibility of premature convergence, while larger populations improved exploration capability at the cost of additional computational overhead. A population size of 100 provided a suitable balance between convergence quality and computational efficiency.

Overall, the convergence behavior demonstrates that the proposed RDA model was capable of gradually refining scheduling solutions while maintaining stable optimization performance across different workload sizes and parameter configurations.

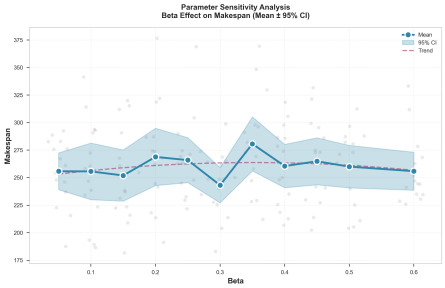
6.4 Parameter Sensitivity Analysis

Multiple experiments were performed by varying one parameter at a time while keeping the remaining parameters fixed. The obtained results were analyzed using makespan, average response time, utilization, convergence behavior, and standard deviation measurements.

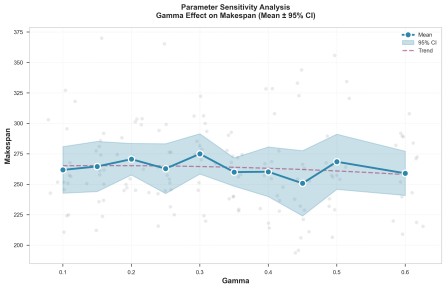
Figure 12 illustrates the impact of different RDA parameter configurations on makespan performance under varying scheduling conditions.



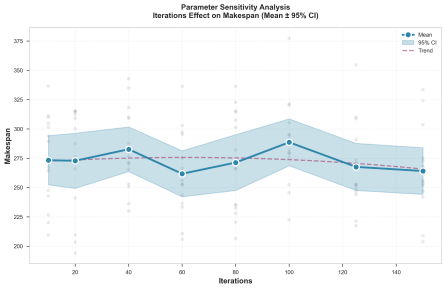
(a) Alpha Sensitivity



(b) Beta Sensitivity



(c) Gamma Sensitivity



(d) Iteration Sensitivity

Fig. 12: Parameter sensitivity analysis using makespan metric

The experimental results indicate that moderate values of α, β, and γ provided comparatively stable scheduling performance by maintaining a balance between exploration and exploitation during optimization. Extreme parameter values either reduced search diversity or increased instability during convergence.

Increasing the number of iterations improved solution refinement during later optimization stages. However, the performance improvement gradually stabilized beyond approximately 62 iterations, indicating diminishing optimization gains after prolonged execution.

Among the evaluated parameter configurations Table 6, $\alpha = 0.35$, $\beta = 0.3$, and $\gamma =$ .45 achieved comparatively lower makespan and response time values. The obtained

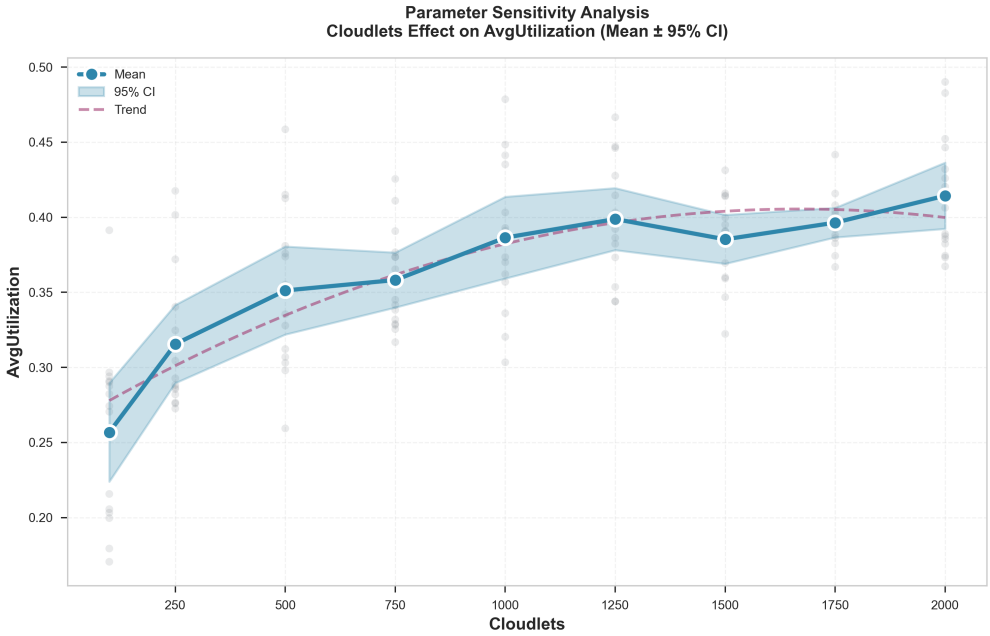


Fig. 13: Resource utilization under increasing cloudlet workloads

results demonstrate that the proposed RDA model maintained stable performance across different parameter settings, indicating robustness of the optimization process.

6.5 Scalability Analysis

Scalability analysis was performed to evaluate the behavior of the proposed model under increasing workload and infrastructure sizes. The experiments included workload configurations ranging from 100 to 2000 cloudlets and heterogeneous cloud infrastructures containing up to 20 hosts and 50 virtual machines.

The obtained results showed that makespan and response time increased gradually as discussed in section 6.1. which indicates effective scalability behavior of the proposed RDA model.

As illustrated in Figure 13, the proposed scheduling mechanism maintained balanced resource utilization across heterogeneous virtual machines under increasing workload conditions. The average utilization improved from approximately 0.26% to 0.42% as workload size increased, demonstrating efficient workload distribution and effective use of available computational resources.

The scalability experiments demonstrate that the proposed RDA-based scheduling mechanism can adapt to larger cloud environments while maintaining stable optimization behavior.

6.6 Statistical Validation

To validate the statistical significance of the obtained scheduling results, multiple independent simulation runs were performed for each evaluated algorithm. The experimental results were analyzed using standard deviation measurements, independent t-tests, and Wilcoxon rank-sum tests.

The statistical significance of the obtained results was evaluated using a significance level of $\alpha = 0.05$. For each pairwise comparison, the null hypothesis H_0 assumed that there was no statistically significant difference between the mean scheduling performances of the compared algorithms, while the alternative hypothesis H_1 assumed that a significant difference existed.

The t-test statistic used for comparison is expressed as:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \tag{27}$$

where $\bar{x}_1$ and $\bar{x}_2$ represent the sample means, s_1^2 and s_2^2 denote the sample variances, and n_1 and n_2 represent the number of observations for the compared algorithms.

Table 7: Statistical Comparison of Scheduling Algorithms

Comparison	**T-Test p-value**	**Wilcoxon p-value**	**Effect Size**
RDA vs PSO	2.99×10^{-7}	2.37×10^{-5}	Large
RDA vs WOA	1.97×10^{-8}	1.42×10^{-6}	Large
RDA vs GA	4.99×10^{-8}	8.01×10^{-8}	Large
RDA vs GWO	9.60×10^{-9}	2.05×10^{-7}	Large
PSO vs WOA	0.3571	0.2894	Small
PSO vs GA	0.0340	0.0549	Medium
PSO vs GWO	0.0677	0.0879	Small
WOA vs GA	0.1550	0.2449	Small
WOA vs GWO	0.3123	0.3085	Small
GA vs GWO	0.6076	0.8236	Negligible

The standard deviation analysis showed that the proposed RDA model maintained it's scheduling performance across repeated simulation runs, indicating robustness of the optimization process. Table 7 presents the statistical comparison between different scheduling algorithms using both parametric and non-parametric significance tests.

For most comparisons involving RDA, the obtained p-values were below the significance threshold of 0.05, indicating statistically significant performance differences. In particular, the comparisons between RDA and GA, GWO, PSO, and WOA produced very small p-values along with large effect sizes, demonstrating that the observed improvements were not caused by random variation alone.

The comparatively larger p-values observed in certain non-RDA comparisons, such as PSO vs WOA and WOA vs GWO, indicate relatively similar scheduling behavior between those approaches. Overall, the statistical analysis supports the reliability and consistency of the proposed RDA-based scheduling mechanism under different workload conditions.

7 Conclusion and Future Work

This study presented an RDA-based scheduling approach for load balancing in cloud computing environments. The experimental results demonstrated that the proposed model achieved competitive scheduling performance by reducing makespan and response time while maintaining balanced resource utilization under heterogeneous workload conditions. The convergence, scalability, and statistical analyses further indicated that the proposed scheduling mechanism maintained stable optimization behavior across varying workload and infrastructure configurations.

The performance improvements primarily resulted from the balanced exploration and exploitation capability of RDA during cloudlet-to-VM allocation. The hierarchical population structure and iterative refinement process enabled the model to gradually improve solution quality while avoiding premature convergence during optimization.

Despite the obtained improvements, the current study was conducted under certain simplifying assumptions. The scheduling model does not currently consider workflow-based task dependencies, dynamic SLA violation handling, or real-time cloud variability present in production cloud environments. In addition, the weighted-sum fitness formulation does not fully capture Pareto-optimal trade-offs between conflicting QoS objectives.

Future work can extend the proposed model by incorporating Pareto-based multi-objective optimization techniques, workflow-aware scheduling mechanisms, and additional fairness metrics such as Jain's Fairness Index [30]. Further investigation involving hybrid metaheuristic approaches, adaptive scheduling strategies, and evaluations may provide additional insights into the robustness and applicability of the proposed model under dynamic cloud conditions.

References

[1] Mell, P., Grance, T.: The nist definition of cloud computing. Technical Report 800-145, National Institute of Standards and Technology, Gaithersburg, MD (September 2011). https://doi.org/10.6028/NIST.SP.800-145 . https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf

[2] Kumar, J., Rani, A., Dhurandher, S.: Convergence of user and service provider perspectives in mobile cloud computing environment: Taxonomy and challenges. International Journal of Communication Systems **33** (2020) https://doi.org/10.1002/dac.4636

[3] Ahmad, I., Bakht, H., Mohan, U.: Cloud computing – a comprehensive definition. Journal of Computing and Management Studies **1**(1), 1–6 (2017)

[4] Kumar, J., Singh, A.: Performance evaluation of metaheuristics algorithms for workload prediction in cloud environment. Applied Soft Computing **113**, 107895 (2021) https://doi.org/10.1016/j.asoc.2021.107895

[5] Shahid, M.A., Islam, N., Alam, M.M., Su'ud, M.M., Musa, S.: A comprehensive study of load balancing approaches in the cloud computing environment and a novel fault tolerance approach. IEEE Access **8**, 130500–130526 (2020) https://doi.org/10.1109/ACCESS.2020.3009184

[6] Alicherry, M., Lakshman, T.V.: Optimizing data access latencies in cloud systems by intelligent virtual machine placement. IEEE Transactions on Parallel and Distributed Systems **25**(3), 647–655 (2013) https://doi.org/10.1109/INFCOM.2013.6566850

[7] Boulmier, A., Abdennadher, N., Chopard, B.: Optimal load balancing and assessment of existing load balancing criteria. Journal of Parallel and Distributed Computing **169**, 211–225 (2022) https://doi.org/10.1016/j.jpdc.2022.07.002

[8] Attallah, S.M.A., Fayek, M.B., Nassar, S.M., Hemayed, E.E.: Proactive load balancing fault tolerance algorithm in cloud computing. Concurrency and Computation: Practice and Experience **33**(10), 6172 (2021) https://doi.org/10.1002/cpe.6172

[9] Mohammadi Golchi, M., Saraeian, S., Heydari, M.: A hybrid of firefly and improved particle swarm optimization algorithms for load balancing in cloud environments: Performance evaluation. Computer Networks **162**, 106860 (2019) https://doi.org/10.1016/j.comnet.2019.106860

[10] Zhou, J., Kumar Lilhore, D., Tao, H., Simaiya, S., Jawawi, D., Alsekait, D., Ahuja, S., Biamba, C., Hamdi, M.: Comparative analysis of metaheuristic load balancing algorithms for efficient load balancing in cloud computing. Journal of Cloud Computing **12** (2023) https://doi.org/10.1186/s13677-023-00453-3

[11] Blum, C., Roli, A.: Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison. Springer, ??? (2003)

[12] Mansouri, N., Zade, B.M.H., Javidi, M.M.: Hybrid task scheduling strategy for cloud computing by modified particle swarm optimization and fuzzy theory. Computers & Industrial Engineering **130**, 597–633 (2019) https://doi.org/10.1016/j.cie.2019.03.006

[13] Li, K., Zomaya, A.Y.: Energy-aware scheduling for cloud computing: A survey. ACM Computing Surveys (CSUR) **51**(3), 1–35 (2018) https://doi.org/10.1145/3186832

[14] Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of ICNN'95 - International Conference on Neural Networks, pp. 1942–19484 (1995).

https://doi.org/10.1109/ICNN.1995.488968

[15] Dorigo, M.: Optimization, learning and natural algorithms (in italian). PhD thesis, Politecnico di Milano, Italy (1992)

[16] Yang, X.-S.: A brief introduction to nature-inspired metaheuristic algorithms. Studies in Computational Intelligence **284**, 1–21 (2010) https://doi.org/10.1007/978-3-642-04944-6_1

[17] Buyya, R., Yeo, C.S., Venugopal, S., Broberg, J., Brandic, I.: Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility. Future Generation Computer Systems **25**(6), 599–616 (2009) https://doi.org/10.1016/j.future.2008.12.001

[18] Fathollahi-Fard, A.M., Hajiaghaei-Keshteli, M., Tavakkoli-Moghaddam, R.: Red deer algorithm (rda): a new nature-inspired meta-heuristic. Soft computing **24**, 14637–14665 (2020)

[19] Sefati, S., Navimipour, N.J.: A qos-aware service composition mechanism in the internet of things using a hidden-markov-model-based optimization algorithm. IEEE Internet of Things Journal **8**(20), 15620–15627 (2021) https://doi.org/10.1109/JIOT.2021.3074499

[20] Pradhan, B., Das, D., Sahoo, S.: An efficient red deer algorithm for multi-objective optimization problems. Journal of Ambient Intelligence and Humanized Computing, 1–17 (2022) https://doi.org/10.1007/s12652-022-03714-w

[21] Goyal, S., Singh, M.: Adaptive and dynamic load balancing in grid using ant colony optimization. International Journal of Engineering and Technology **4** (2012)

[22] Kruekaew, B., Kimpan, W.: Enhancing of artificial bee colony algorithm for virtual machine scheduling and load balancing problem in cloud computing. International Journal of Computational Intelligence Systems **13** (2020) https://doi.org/10.2991/ijcis.d.200410.002

[23] Sefati, S.S., Mousavinasab, M., Farkhady, R.: Load balancing in cloud computing environment using the grey wolf optimization algorithm based on the reliability: performance evaluation. The Journal of Supercomputing **78** (2022) https://doi.org/10.1007/s11227-021-03810-8

[24] Anbarkhan, S.H., Rakrouki, M.A.: An enhanced pso algorithm for scheduling workflow tasks in cloud computing. Electronics (2023)

[25] Lilhore, U.K., Simaiya, S., Prajapati, Y.N., Rai, A.K., Ghith, E.S., Tlija, M., Lamoudan, T., Abdelhamid, A.A.: A multi-objective approach to load balancing in cloud environments integrating ACO and WWO techniques. Scientific Reports

15, 12036 (2025) https://doi.org/10.1038/s41598-025-96364-1

[26] Shi, L., Wang, X., Ma, R.T., Tay, Y.C.: Weighted fair caching: Occupancy-centric allocation for space-shared resources. ACM SIGMETRICS Performance Evaluation Review **46**(3), 35–36 (2019)

[27] Zanbouri, K., Navimipour, N.: A cloud service composition method using a trust-based clustering algorithm and honeybee mating optimization algorithm. International Journal of Communication Systems **33** (2019) https://doi.org/10.1002/dac.4259

[28] Calheiros, R.N., Ranjan, R., Beloglazov, A., De Rose, C.A., Buyya, R.: Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Software: Practice and experience **41**(1), 23–50 (2011)

[29] Goyal, T., Singh, A., Agrawal, A.: Cloudsim: simulator for cloud computing infrastructure and modeling. Procedia Engineering **38**, 3566–3572 (2012) https://doi.org/10.1016/j.proeng.2012.06.412 . INTERNATIONAL CONFERENCE ON MODELLING OPTIMIZATION AND COMPUTING

[30] Rezaeinia, N., Góez, J.C., Guajardo, M.: On efficiency and the jain's fairness index in integer assignment problems. Computational Management Science **20**(1), 42 (2023)